



GLK Transit

Admin UI

Technical Documentation

Version: 1.0
Date: May 25, 2026
Organization: Grand Lake Municipality
Classification: Internal — Confidential

Table of Contents

1.	Project Overview
2.	Technology Stack
3.	System Architecture
4.	Application Routes & Pages
5.	Component Library
6.	API Layer & Backend Integration
7.	Authentication & Authorization
8.	State Management
9.	Real-Time Features
10.	Key Features
11.	Data Types & Models
12.	Configuration & Environment
13.	Styling & Design System
14.	Developer Guide
15.	Security Considerations
16.	Appendix

1. Project Overview

The GLK Transit Admin UI is the primary administrative dashboard for managing public transit operations within the Grand Lake Municipality. It provides transit authority staff with a centralized, web-based interface to monitor fleet status, manage drivers and routes, handle ticketing and payments, and administer system-wide role-based permissions.

Document Purpose

This document describes the technical architecture, component structure, API integration patterns, and operational configuration of the GLK Transit Admin UI. It is intended for software developers, system architects, and technical operations staff responsible for maintaining and extending the platform.

Project Identifiers

Field	Value
Project Name	GLK Transit Admin UI
Repository	glktransit-admin-ui
Version	0.1.0
Status	Active Development
Organization	Grand Lake Municipality — Transit Operations Division
API Base URL	https://api.grandlakemunicipality.ca/api/v1/
Target Environment	Production (DigitalOcean Spaces for assets)

Core Capabilities at a Glance

- Real-time fleet monitoring and bus operations dashboard
- Full CRUD management for buses, maintenance records, and mechanics
- Driver lifecycle management — registration, shift assignment, incident reporting
- Route and schedule management with stop-level configuration
- Ticket catalog management and payment transaction history
- Refund processing and pass issuance workflows
- Role-based admin management with a visual permission matrix
- Comprehensive audit trails and login log reporting
- WebSocket-driven real-time notifications
- Responsive, mobile-compatible interface

2. Technology Stack

Core Framework

Technology	Version	Role
Next.js	16.2.1	Full-stack React framework (App Router)
React	19.2.4	UI component library
TypeScript	5.x	Typed JavaScript superset
Node.js	LTS	Runtime environment

UI & Styling

Library	Version	Purpose
TailwindCSS	4.x	Utility-first CSS framework with CSS variable theme
shadcn/ui	Latest	Pre-built accessible component library (Base-Mira style)
Base-UI React	1.3.0	Low-level accessible UI primitives
Lucide React	Latest	Primary icon set
Hugeicons	Latest	Secondary icon set
Class Variance Authority	Latest	Component variant management
Tailwind Merge	Latest	Conflict-safe Tailwind class merging
tw-animate-css	Latest	Animation utilities

Data & Communication

Library	Version	Purpose
Axios	1.14.0	HTTP client with request/response interceptors
Socket.io-client	4.8.3	WebSocket client for real-time notifications
Zustand	5.0.12	Lightweight global state management with persistence
jose	Latest	JWT signing and verification for session tokens

Utilities

Library	Version	Purpose
Sonner	2.0.7	Toast notification system
React Phone Number Input	3.4.16	Formatted international phone fields
Geist / Work Sans / Inter	Next.js Font	Optimized web fonts

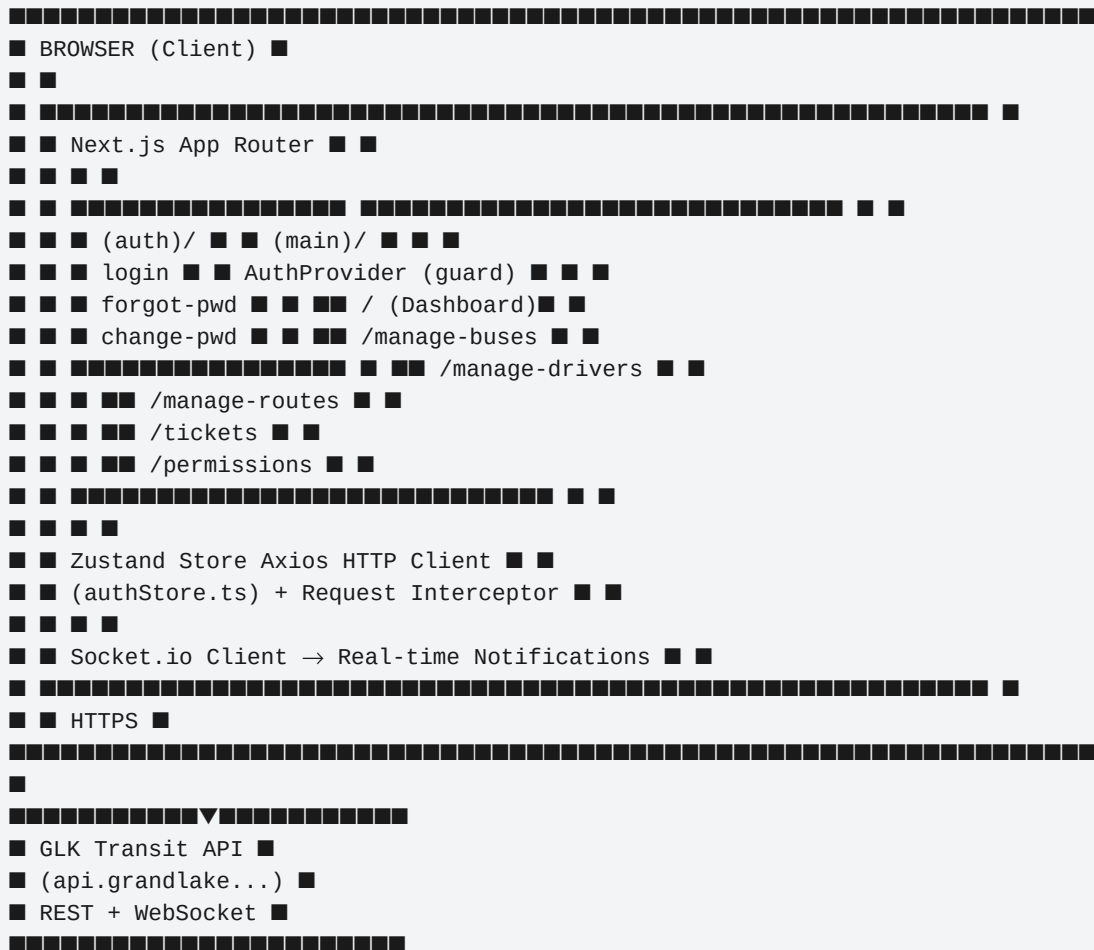
Development Tooling

Tool	Purpose
ESLint 9 + Next.js config	Static analysis and code quality
PostCSS 4	CSS transformation pipeline
TypeScript strict mode	Compile-time type safety
Next.js Image Optimization	Automatic image resizing and CDN caching

3. System Architecture

The application follows the Next.js App Router architecture with a clear separation between authentication routes, protected main application routes, server-side API utilities, and a rich client-side component layer.

High-Level Architecture Diagram



Directory Structure

```
glktransit-admin-ui/
├── app/ # Next.js App Router root
│   ├── (auth)/ # Public auth routes (no sidebar)
│   │   ├── login/
│   │   ├── forgot-password/
│   │   └── change-password/
│   └── (main)/ # Protected routes (sidebar + topbar)
│       ├── layout.tsx # Shell with AuthProvider, Sidebar, Topbar
│       ├── page.tsx # Dashboard
│       └── manage-buses/
```

- ■ ■■■ manage-drivers/
- ■ ■■■ manage-routes/
- ■ ■■■ tickets/
- ■ ■■■ permissions/
- ■■■ layout.tsx # Root HTML shell + font providers
- ■■■ globals.css # CSS variable theme + global resets
-
- components/ # React components
- ■■■ ui/ # shadcn primitives (25+ files)
- ■■■ manage-buses/ # Bus & maintenance components
- ■■■ manage-drivers/ # Driver management components
- ■■■ manage-routes/ # Route management components
- ■■■ tickets/ # Ticket & payment components
- ■■■ permissions/ # Admin/role/audit components
- ■■■ authProvider.tsx # Session guard
- ■■■ sidebar.tsx
- ■■■ topbar.tsx
- ■■■ notificationModal.tsx
-
- lib/
- ■■■ api/ # Typed Axios wrappers (10 modules)
- ■■■ stores/ # Zustand global stores
- ■■■ types/ # TypeScript interfaces & types
- ■■■ hooks/ # Custom React hooks
- ■■■ utils/ # Formatting & helper utilities
- ■■■ constants/ # App-wide constants
- ■■■ client.ts # Axios instance configuration
- ■■■ config.ts # Runtime configuration
-
- public/assets/ # Static images & logos
- .env # Environment variables
- next.config.ts
- tailwind.config.ts
- tsconfig.json

4. Application Routes & Pages

Route Map

Route	File	Protected	Description
/login	app/(auth)/login/page.tsx	No	Email/password login form
/forgot-password	app/(auth)/forgot-password/page.tsx	No	Initiate password reset flow
/change-password	app/(auth)/change-password/page.tsx	No	Set new password after reset
/	app/(main)/page.tsx	Yes	Dashboard: KPIs, fleet overview, alerts
/manage-buses	app/(main)/manage-buses/page.tsx	Yes	Bus CRUD, maintenance, mechanics
/manage-drivers	app/(main)/manage-drivers/page.tsx	Yes	Driver profiles, shifts, incidents
/manage-routes	app/(main)/manage-routes/page.tsx	Yes	Route CRUD, stop configuration, schedules
/tickets	app/(main)/tickets/page.tsx	Yes	Ticket catalog, transactions, refunds, passes
/permissions	app/(main)/permissions/page.tsx	Yes	Admin users, roles, permission matrix, audit logs

Route Groups & Layouts

Next.js route groups (parenthesized folder names) share layout files without adding URL segments:

- **(auth)**: Minimal layout — centered card, no navigation. Applies to login, forgot-password, and change-password pages.
- **(main)**: Full shell layout — AuthProvider session guard wraps a persistent Sidebar and Topbar. All protected routes live here.

Dashboard Page Detail

The root dashboard page is the primary entry point for authenticated users. It aggregates live metrics and operational data:

- Four KPI summary cards: Total Ridership, Ticket Sales, Revenue Collected, Ticket Validations — each with period comparison
- Tabbed bus operations table (All / Active / Inactive) with inline map modal for live bus location
- Time period filter: Today, This Week, This Month, This Year
- Real-time alerts sidebar showing system warnings (e.g. maintenance overdue, route anomalies)

- Personalized greeting using the authenticated admin's display name from Zustand store

5. Component Library

Layout & Shell Components

Component	File	Responsibility
AuthProvider	components/authProvider.tsx	Validates session on mount; redirects to /login on failure; shows spinner during check
Sidebar	components/sidebar.tsx	Fixed navigation with links to all main sections; active link highlighting; user profile logo; collapsible on mobile
Topbar	components/topbar.tsx	Search bar; notification bell with modal; user profile dropdown with logout
NotificationModal	components/notificationModal.tsx	Lists real-time and historical notifications from the API; uses Socket.io hook
BusMap	components/bus-map.tsx	Map visualization for live bus location display

Bus Management Components

Component	Responsibility
BusList	Paginated, searchable, filterable table of all buses with status badges and action menus
BusForm	Add/edit modal: tracking ID, bus number, route assignment, bus type, accessibility features
MaintenanceManagement	View and manage maintenance records linked to a specific bus
MaintenanceLogs	History view of all maintenance work performed by mechanics
MaintenanceForm	Create/edit maintenance record with type, date, notes, cost
AddMechanicModal	Register a new mechanic with name and contact details
DeleteBusModal	Confirmation dialog before permanently removing a bus record

Driver Management Components

Component	Responsibility
DriverList	Paginated driver table with search, date-range filter, status filter, and action menu
DriverForm	Register/edit driver: name, email, phone, staff ID, assigned route, date of birth

Component	Responsibility
DriverProfile	Detailed view: avatar, personal info, document verification status, incident history
AssignShiftModal	Assign driver to a bus/route combination for a given shift period
IncidentReportModal	File an incident report with description, date, severity, and optional file attachments

Route Management Components

Component	Responsibility
RouteList	Filterable list of all transit routes with creation date and action menu
RouteForm	Create/edit route with stop names, distances between stops, and estimated travel time
ScheduleModal	View and manage timed schedules associated with a route
DeleteRouteModal	Confirmation dialog for route deletion

Ticket & Payment Components

Component	Responsibility
TransactionList	Full paginated transaction history with status filters and receipt download
TicketTypeList	Manage the ticket catalog — single rides and period passes by rider type and route
RefundManagement	Process and review refund requests; approve or reject with notes
PassManagement	Issue/revoke transit passes (UI present, currently disabled pending backend)
RefundSettings	Configure automatic refund rules and thresholds

Permissions Components

Component	Responsibility
ManageAdminsView	List all admin users; invite new admins; activate/deactivate accounts; reassign roles
ManageRolesView	Visual permission matrix editor: role × menu item × action → toggle boolean access
AuditTrailsTable	Searchable, paginated log of all admin actions with timestamp, actor, and action type

Component	Responsibility
LoginLogsTable	Admin login history showing timestamp, IP address, device, and role at login time
PermissionsDetailModal	Displays which roles currently have access to a selected system action

UI Primitives (shadcn/ui)

- Button, Input, Textarea, Label — form controls
- Card, CardHeader, CardContent, CardFooter — content containers
- Dialog, DialogTrigger, DialogContent — modal dialogs
- Table, TableHeader, TableBody, TableRow, TableCell — data grids
- Tabs, TabsList, TabsTrigger, TabsContent — tabbed interfaces
- Badge — status indicators
- Select, DropdownMenu, Checkbox, Switch — selection controls
- Avatar — user/entity profile images with fallback initials
- Pagination — page navigation
- ScrollArea — custom scrollable regions
- PhoneInput — formatted international phone number input

6. API Layer & Backend Integration

Axios Client Configuration

A single Axios instance (**lib/client.ts**) is shared across all API modules. It is pre-configured with:

- Base URL from NEXT_PUBLIC_API_BASE_URL environment variable
- 30-second request timeout
- Request interceptor: reads the auth_token_client cookie and injects Authorization: Bearer <token> on every outbound request
- Response interceptor: normalizes error payloads from the backend and handles HTTP error codes (401 → session expired, 403 → forbidden, 404 → not found, 422 → validation error, 500 → server error)

API Modules

Module	File	Key Endpoints
Auth	lib/api/auth.ts	login(), getUser(), logout() — server-side only (uses jose JWT)
Fleet	lib/api/fleet.ts	getBuses(), addBus(), updateBus(), deleteBus(), createMaintenance(), updateMaintenance(), assignShift(), createMechanic(), getMechanics()
Drivers	lib/api/drivers.ts	registerDriver(), getAllDrivers(), getDriver(), updateDriver(), deleteDriver(), uploadDriverDocuments(), uploadDriverPicture()
Routes	lib/api/routes.ts	createRoute(), getAllRoutes(), updateRoute(), deleteRoute()
Tickets	lib/api/tickets.ts	getAllTicketCatalog(), createTicketEntry(), updateTicketEntry(), deleteTicketEntry()
Transactions	lib/api/transactions.ts	getTransactionOrders(), getTransactionById(), createTransactionOrders()
Admin	lib/api/admin.ts	registerAdmin(), getAllAdmins(), getAllLoginLogs(), getAllAuditTrails(), getAllRoleMatrix(), updateRoleMatrix(), deactivateAdmin(), activateAdmin(), reassignAdminRole(), revokeAdminInvitation(), resendAdminInvitation()
Dashboard	lib/api/dashboard.ts	getDashboardOverview() — KPI metrics by time period
Notifications	lib/api/notifications.ts	Fetch notification history
User	lib/api/user.ts	User profile utilities

File Upload Pattern

Document and image uploads use the multipart/form-data encoding. API functions such as uploadDriverDocuments() and uploadBusDocuments() accept a FormData object and forward it

directly to the backend. Supported upload types include:

- Driver profile pictures (JPEG/PNG)
- Driver licence and certification documents (PDF/JPEG)
- Bus registration and compliance documents
- Incident report attachments

Pagination & Filtering

List endpoints consistently accept query parameters for server-side pagination and filtering:

Parameter	Type	Description
page	number	Current page index (1-based)
limit	number	Number of records per page
search	string	Full-text search across relevant fields
status	string	Filter by entity status (active/inactive/pending)
startDate / endDate	ISO date string	Date range filter for time-bounded queries

Error Handling Convention

All API calls are wrapped in try/catch blocks. On error, the Axios response interceptor normalizes the error message. Components display errors via Sonner toast notifications. Backend validation errors (HTTP 422) expose per-field messages that are mapped to form error states.

7. Authentication & Authorization

Authentication Flow

1. Admin navigates to /login
2. Submits credentials (email + password)
3. Client calls → POST /auth/login (via lib/api/auth.ts)
4. Backend returns { token, user, profile }
5. Server action signs a JWT using jose (HS256, JWT_SECRET)
6. Two cookies are set:
 - auth_token → HttpOnly, secure, sameSite:lax (1 day)
 - auth_token_client → readable, sameSite:lax (1 day)
7. Zustand authStore.setUser({ user, profile }) is called
8. Admin is redirected to / (Dashboard)

On every protected page load:

9. AuthProvider calls getUser()
10. getUser() reads auth_token cookie, verifies JWT
11. On success → session valid, page renders
12. On failure → cookies cleared, redirect to /login

Session Token Architecture

Cookie	Accessible By	Purpose
auth_token	Server only (HttpOnly)	Used by server-side getUser() for JWT verification; inaccessible to JavaScript
auth_token_client	Client JavaScript	Read by Axios request interceptor to inject Bearer token into API requests

Role-Based Authorization

The system uses a backend-enforced role-based access control (RBAC) model. The Admin UI provides a visual management interface for the permission matrix but does not perform client-side enforcement — all authorization decisions are made by the backend API.

Roles

- **superAdmin**: Full system access including permission matrix management
- **admin**: General administrative access to operational modules
- **operationsAdmin**: Access to fleet, routes, and driver management
- **supportStaff**: Read-only or limited-write access to tickets and transactions

Permission Matrix Structure

The matrix is a three-dimensional mapping: **Role × Menu Item × Action** → **Boolean**. It is stored in the backend database and fetched via getAllRoleMatrix(). Admins with appropriate permissions can

toggle individual cells and persist changes via `updateRoleMatrix(records)`.

8. State Management

Zustand Auth Store

Global application state is minimal. The primary global store is **lib/stores/authStore.ts**, implemented with Zustand and the persistence middleware:

```
interface AuthState {
  user: UserData | null; // { user: User, profile: Profile }
  _hasHydrated: boolean; // True after localStorage rehydration
  setUser: (user: UserData) => void;
  clearAuth: () => void;
  setHasHydrated: (state: boolean) => void;
}

// Persisted to localStorage key: "auth-user"
// Only the 'user' field is persisted across page refreshes
```

The `_hasHydrated` flag prevents flash-of-unauthenticated-content by allowing components to defer rendering until the persisted state has been loaded from `localStorage`.

Local Component State

All other state is managed locally within components using React's `useState` and `useCallback` hooks:

- List filters, search query, and pagination index in list components
- Form field values and validation errors in form components
- Modal open/close state in parent page components
- Loading and error flags for async data fetch operations
- Selected row/entity for detail or edit operations

Data Fetching Pattern

```
// Typical data fetch pattern used across the application
const [data, setData] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

const fetchData = useCallback(async () => {
  setLoading(true);
  try {
    const res = await apiModule.getAll({ page, limit, search });
    setData(res.data);
  } catch (err) {
    toast.error(err.message);
  } finally {
    setLoading(false);
  }
}, [page, limit, search]);
```

```
useEffect(() => { fetchData(); }, [fetchData]);
```

9. Real-Time Features

WebSocket Notification System

The application maintains a persistent WebSocket connection using Socket.io to receive real-time notifications from the backend. This is implemented via a custom hook at `lib/hooks/useNotificationSocket.ts`.

Connection Configuration

Setting	Value
Server URL	NEXT_PUBLIC_API_BASE_URL (same as REST API)
Transport	WebSocket only (no long-polling fallback)
Authentication	JWT token passed in Socket.io handshake auth object
Reconnection attempts	5 (then gives up)
Event subscribed	notification:new

Usage

```
// lib/hooks/useNotificationSocket.ts
const socket = io(baseUrl, {
  auth: { token }, // JWT from auth_token_client cookie
  transports: ["websocket"],
  reconnectionAttempts: 5,
});

socket.on("notification:new", (notification: Notification) => {
  onNewNotification(notification); // callback updates topbar badge
});

// Component usage (Topbar)
useNotificationSocket({
  token,
  onNewNotification: (n) => setNotifications(prev => [n, ...prev]),
});
```

Notification Types

Notifications are displayed in the Topbar notification modal and may include:

- Fleet alerts (maintenance overdue, bus offline)
- Driver incident reports filed
- Refund requests requiring attention
- System-level administrative alerts
- Shift assignment confirmations

10. Key Features

Feature	Module	Details
Dashboard Analytics	Dashboard (/)	KPI cards (ridership, sales, revenue, validations) with period comparison; bus operations table; real-time alert sidebar
Fleet Management	Manage Buses	CRUD operations for buses; route assignment; bus type & accessibility configuration; status tracking (online/offline); document uploads
Maintenance Tracking	Manage Buses	Log maintenance events with type, date, notes, and cost; assign mechanics; view full maintenance history per bus
Driver Management	Manage Drivers	Driver registration with staff ID; profile documents and picture upload; shift assignment to bus/route; incident reporting with attachments
Route Management	Manage Routes	Create routes with ordered stops, inter-stop distances, and estimated travel time; schedule management per route
Ticket Catalog	Tickets	Define ticket types by rider category (adult, student, senior), route applicability, price, and validity period
Transaction History	Tickets	View all payment transactions with status (completed, pending, failed); search and date filter; downloadable receipt
Refund Processing	Tickets	Review refund requests; approve or reject with written justification; configure automatic refund rules
Admin Management	Permissions	Register new admins via direct creation or invitation email; activate/deactivate accounts; reassign roles
Permission Matrix	Permissions	Interactive grid editor mapping roles to menu-item/action combinations; changes saved immediately to backend
Audit Logs	Permissions	Immutable log of all admin actions with actor identity, action type, timestamp, and affected resource
Login Logs	Permissions	Login history for all admin accounts including IP address, device, timestamp, and role at time of login
Real-Time Notifications	Topbar	Socket.io-driven notification badge and modal for instant operational alerts
Responsive Design	All	Mobile-first layout; collapsible sidebar on small screens; adaptive data grids

11. Data Types & Models

Auth Types (lib/types/auth.ts)

```
interface User {
  id: string;
  email: string;
  role: "superAdmin" | "admin" | "operationsAdmin" | "supportStaff";
  isActive: boolean;
  createdAt: string;
}

interface Profile {
  id: string;
  firstName: string;
  lastName: string;
  phone?: string;
  avatarUrl?: string;
}

interface UserData { user: User; profile: Profile; }
```

Bus Types (lib/types/buses.ts)

```
interface Bus {
  id: string;
  trackingId: string;
  busNumber: string;
  routeId: string;
  type: "standard" | "articulated" | "minibus";
  isAccessible: boolean;
  status: "active" | "inactive" | "maintenance";
  documents: BusDocument[];
}

interface Maintenance {
  id: string;
  busId: string;
  type: string;
  date: string;
  notes: string;
  cost: number;
  mechanicId: string;
}

interface Mechanic {
  id: string;
  name: string;
  phone: string;
  email: string;
}
```

Driver Types (lib/types/drivers.ts)

```
interface Driver {
  id: string;
  staffId: string;
  firstName: string;
  lastName: string;
  email: string;
  phone: string;
  dateOfBirth: string;
  routeId?: string;
  status: "active" | "inactive" | "suspended";
  profilePictureUrl?: string;
  documents: DriverDocument[];
  incidents: Incident[];
}
```

Route Types (lib/types/routes.ts)

```
interface Route {
  id: string;
  name: string;
  stops: Stop[];
  estimatedDuration: number; // minutes
  createdAt: string;
}

interface Stop {
  name: string;
  order: number;
  distanceFromPrevious: number; // km
}

interface Schedule {
  id: string;
  routeId: string;
  departureTime: string;
  arrivalTime: string;
  days: string[];
}
```

Ticket & Transaction Types (lib/types/tickets.ts)

```
interface TicketCatalogEntry {
  id: string;
  name: string;
  riderType: "adult" | "student" | "senior" | "child";
  routeId?: string;
  price: number;
  validityDays?: number;
  type: "single" | "pass";
  isActive: boolean;
}
```

```
}

interface Transaction {
  id: string;
  ticketId: string;
  userId: string;
  amount: number;
  status: "completed" | "pending" | "failed" | "refunded";
  createdAt: string;
  receiptUrl?: string;
}
```

Permissions Types (lib/types/permissions.ts)

```
interface AdminUser {
  id: string;
  email: string;
  role: Role;
  isActive: boolean;
  inviteStatus?: "pending" | "accepted";
  createdAt: string;
}

interface RoleMatrixRecord {
  roleId: string;
  menuItemId: string;
  actionId: string;
  hasPermission: boolean;
}

interface AuditTrail {
  id: string;
  adminId: string;
  action: string;
  resource: string;
  timestamp: string;
  ipAddress: string;
}
```

12. Configuration & Environment

Environment Variables

Variable	Required	Description
NEXT_PUBLIC_API_BASE_URL	Yes	Base URL for the backend REST API and WebSocket server (e.g. https://api.grandlakemunicipality.ca/api/v1/)
JWT_SECRET	Yes (server-side)	Hex-encoded secret used by jose to sign and verify session JWTs. Must be at least 256 bits.
NODE_TLS_REJECT_UNAUTHORIZED	Dev only	Set to 0 in development environments with self-signed certificates. Must NOT be set in production.

Next.js Configuration (next.config.ts)

```
// Configured remote image patterns for Next.js Image Optimization:
remotePatterns: [
  { hostname: "picsum.photos" },
  { hostname: "api-dev.grandlakemunicipality.ca" },
  { hostname: "/*.digitaloceanspaces.com" },
]
```

TypeScript Configuration (tsconfig.json)

Setting	Value
target	ES2017
strict	true (full strict mode)
moduleResolution	bundler
Path alias @/*	Root directory — import @/lib/api/auth resolves from project root
jsx	preserve (Next.js handles JSX transform)

13. Styling & Design System

CSS Variable Theme

The application uses a token-based CSS variable system defined in **app/globals.css**. All components reference these variables, enabling consistent theming and straightforward dark mode support.

Token Group	Variables	Purpose
Brand	--color-brand-primary (#40843c), --color-brand-dark (#2d6b2a)	Primary green brand color and darker variant for hover/focus states
Content	--color-content-primary, --color-content-secondary, --color-content-tertiary	Text color hierarchy from high-contrast to muted
Surface	--color-surface-primary, --color-surface-secondary, --color-surface-elevated	Background layer colors for cards, modals, and page backgrounds
Status	--color-error-*, --color-warning-*, --color-info-*, --color-success-*	Semantic color groups for feedback states
Border	--color-border-default, --color-border-strong	Stroke colors for dividers and component borders

Typography

Font	Usage	Loading
Work Sans	Primary UI font — headings, labels, body text	next/font/google, variable font, preloaded
Inter	Secondary UI font — data tables, code-adjacent text	next/font/google, variable font, preloaded
Geist	Tertiary / monospace alternative	next/font/google

Responsive Breakpoints (TailwindCSS defaults)

Breakpoint	Min Width	Typical Use
sm	640px	Compact single-column layouts, small modal widths
md	768px	Two-column grid layouts, visible sidebar toggle
lg	1024px	Full sidebar always visible, multi-column grids

Breakpoint	Min Width	Typical Use
xl	1280px	Wider content areas, additional data table columns

Utility Composition Pattern

```
// lib/utils/utils.ts
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// Usage in components:
<button className={cn(
  "base-classes",
  variant === "primary" && "primary-classes",
  disabled && "disabled-classes",
  className // allow external override
)} />
```

14. Developer Guide

Prerequisites

Requirement	Version	Notes
Node.js	18+ LTS	Required for Next.js 16 and React 19
npm / yarn / pnpm	Latest	Package manager of choice
Git	Any	Source control
GLK Transit API	Running instance	Either local or pointing to staging/production via NEXT_PUBLIC_API_BASE_URL

Local Development Setup

```
# 1. Clone the repository
git clone <repository-url>
cd glktransit-admin-ui

# 2. Install dependencies
npm install

# 3. Configure environment
cp .env.example .env
# Edit .env:
# NEXT_PUBLIC_API_BASE_URL=https://api.grandlakemunicipality.ca/api/v1/
# JWT_SECRET=<your-256-bit-hex-secret>
# NODE_TLS_REJECT_UNAUTHORIZED=0 # dev only

# 4. Start development server
npm run dev
# App available at http://localhost:3000
```

Available Scripts

Script	Command	Description
Development	npm run dev	Start Next.js dev server with hot reload on http://localhost:3000
Build	npm run build	Compile and optimize for production
Start	npm run start	Serve the production build locally
Lint	npm run lint	Run ESLint across the codebase

Adding a New Feature Module

Follow these conventions when adding a new feature area:

- Create a new page at `app/(main)/<feature-name>/page.tsx`
- Add navigation link to `components/sidebar.tsx`
- Create feature components in `components/<feature-name>/`
- Add typed API functions in `lib/api/<feature>.ts`
- Add TypeScript interfaces in `lib/types/<feature>.ts` and export from `lib/types/index.ts`
- Use the `cn()` utility for all dynamic class composition
- Display all async errors with `toast.error()` (Sonner)
- Use `async/await` with `try/catch` — do not use `.then()` chains

Code Conventions

Convention	Detail
Client components	Add 'use client' directive at the top of any component using hooks, browser APIs, or event handlers
Server functions	Authentication utilities in <code>lib/api/auth.ts</code> use 'use server' and run exclusively in the Node.js runtime
Async data fetching	Use <code>useCallback</code> + <code>useEffect</code> pattern; always handle loading and error states
Search debouncing	Debounce user search inputs at 400ms before triggering API calls
Error feedback	All user-facing errors displayed via Sonner toast, never <code>alert()</code> or <code>console.log()</code> only
Modal pattern	Modals controlled by parent-level <code>isOpen</code> boolean state; callbacks for <code>onClose</code> and <code>onSuccess</code>
Icon imports	Prefer Lucide React for consistency; Hugeicons for specialized icons listed in <code>components.json</code>

15. Security Considerations

Session Security

- **HttpOnly cookies:** The primary `auth_token` cookie is `HttpOnly`, preventing JavaScript access and mitigating XSS-based token theft.
- **Short-lived sessions:** JWT tokens expire after 24 hours, limiting the window of exposure for stolen tokens.
- **Secure flag:** Both session cookies are marked `secure: true` in production, ensuring they are only transmitted over HTTPS.
- **SameSite: Lax:** Provides protection against most CSRF attacks while allowing top-level navigation.

Backend Authorization

The Admin UI relies entirely on the backend API for authorization enforcement. Every API request carries the Bearer token and the backend validates permissions server-side for each operation. The client-side permission matrix UI is informational only and does not gate access in the browser.

Production Checklist

Item	Status / Action Required
TLS certificate	Ensure HTTPS is enforced; remove <code>NODE_TLS_REJECT_UNAUTHORIZED=0</code>
JWT_SECRET strength	Use a cryptographically random 256-bit (32-byte) hex secret; rotate periodically
API base URL	Must be HTTPS in production
CORS policy	Backend must restrict allowed origins to the deployed Admin UI domain
Content Security Policy	Add CSP headers via Next.js middleware to restrict script/style sources
Dependency audit	Run <code>npm audit</code> before each production deployment
Error logging	Integrate a server-side error tracking service (e.g. Sentry) for production visibility

Known Limitations

- Client-side RBAC is not enforced — UI elements are not hidden by role; the backend API must enforce all access control.
- The `NODE_TLS_REJECT_UNAUTHORIZED=0` setting in `.env` must be removed before production deployment.
- Pass Management feature is present in the UI but disabled pending full backend implementation.

- The search bar in the Topbar is a UI placeholder with no connected search functionality at this time.

16. Appendix

A. Complete Dependency List

Package	Version	Category
next	16.2.1	Framework
react	19.2.4	Framework
react-dom	19.2.4	Framework
typescript	5.x	Language
tailwindcss	4.x	Styling
@base-ui-components/react	1.3.0	UI
class-variance-authority	latest	UI
tailwind-merge	latest	UI
clsx	latest	UI
lucide-react	latest	Icons
@hugeicons/react	latest	Icons
tw-animate-css	latest	Animation
axios	1.14.0	HTTP
socket.io-client	4.8.3	WebSocket
zustand	5.0.12	State
jose	latest	Auth
sonner	2.0.7	Notifications
react-phone-number-input	3.4.16	Forms
eslint	9.x	DevTools
postcss	4.x	DevTools

B. API Error Codes

HTTP Status	Meaning	Admin UI Behavior
200 OK	Success	Data rendered; success toast on mutations
201 Created	Resource created	Success toast; list refreshed

HTTP Status	Meaning	Admin UI Behavior
400 Bad Request	Invalid request payload	Error toast with message from backend
401 Unauthorized	Token invalid/expired	Session cleared; redirect to /login
403 Forbidden	Insufficient permissions	Error toast; no redirect
404 Not Found	Resource does not exist	Error toast
422 Unprocessable Entity	Validation failure	Field-level error messages displayed in form
500 Internal Server Error	Backend error	Generic error toast

C. External Services

Service	Purpose	Configuration
Grand Lake Transit API	Primary REST + WebSocket backend	NEXT_PUBLIC_API_BASE_URL
DigitalOcean Spaces	Static asset storage (bus/driver documents, images)	Configured as Next.js remote image hostname
picsum.photos	Placeholder images in development	Configured as Next.js remote image hostname

D. Glossary

Term	Definition
App Router	Next.js 13+ routing system based on the file system within the app/ directory
AuthProvider	React component that guards protected routes by validating the user session
RBAC	Role-Based Access Control — restricts system access based on assigned user roles
Permission Matrix	Three-dimensional mapping (Role × Menu Item × Action) that defines access rights
Zustand	Lightweight React state management library with persistence middleware support
jose	JavaScript library for JSON Web Token (JWT) signing and verification in Node.js
Socket.io	Library enabling real-time bidirectional event-based communication over WebSockets

Term	Definition
shadcn/ui	Open-source component collection built on Radix UI primitives with Tailwind styling
CVA	Class Variance Authority — utility for defining component style variants
Sonner	Toast notification library for React with a clean accessible API
KPI	Key Performance Indicator — dashboard summary metrics (ridership, revenue, etc.)
Audit Trail	Immutable, timestamped log of administrative actions for compliance and accountability

Document generated on May 25, 2026 • Grand Lake Municipality — Transit Operations Division • GLK Transit Admin UI v0.1.0